

文件包含漏洞分析与修复报告

Flask 用户管理系统

实验名称：文件包含漏洞分析与修复

实验日期：2026 年 7 月 13 日

1 项目概述

1.1 项目简介

本项目是一个基于 Python Flask 框架的用户信息管理系统，提供用户登录、注册、个人中心、充值、搜索、头像上传和动态页面加载等功能。本次实验针对 动态页面加载功能 中的文件包含漏洞进行安全审计与修复。

1.2 涉及文件访问的功能

功能	路由	涉及文件操作
动态页面加载	GET /page	os.path.join() + open() 读取文件

1.3 存在文件包含漏洞的位置

经过对项目源代码的逐行审计，本次实验发现 **1** 处 文件包含漏洞：

漏洞编号	文件	函数	路由	风险等级
FL-001	app.py	dynamic_page()	GET /page	严重

1.4 修复方式概述

- 建立文件白名单（ALLOWED_PAGES），只允许 help、about、index 等预设页面
- 使用 os.path.normpath() 对路径进行规范化处理
- 校验最终路径必须位于 pages/ 目录内
- 统一返回错误信息，不泄露服务器路径

2 文件包含漏洞概述

2.1 什么是文件包含漏洞

文件包含漏洞（File Inclusion Vulnerability）是指 Web 应用程序在读取文件时，使用用户可控的参数来构建文件路径，且没有对路径进行充分的安全校验，导致攻击者可以读取服务器上的任意文件。

2.2 为什么会产生文件包含漏洞

产生文件包含漏洞的根源是：应用程序信任了用户输入的文件路径。程序使用用户输入的参数直接拼接文件路径，没有检查路径中是否包含 `../` 等目录遍历字符，也没有限制文件读取的范围。

2.3 本项目存在漏洞的原因

动态页面加载功能的路由 `/page` 从 URL 参数 `name` 获取页面名称，然后使用 `os.path.join("pages", name)` 拼接文件路径，最后用 `open()` 读取文件内容。代码如下：

```
@app.route("/page")
def dynamic_page():
    name = request.args.get("name", "")
    ...
    page_path = os.path.join("pages", name)
    if os.path.isfile(page_path):
        with open(page_path, "r", encoding="utf-8") as f:
            page_content = f.read()
    else:
        page_path_html = page_path + ".html"
        if os.path.isfile(page_path_html):
            with open(page_path_html, "r", encoding="utf-8") as f:
                page_content = f.read()
```

这段代码存在三个问题：

- 无白名单：任何文件名都可以传入，没有限定允许访问的文件范围
- 无路径校验：没有检查 `name` 中是否包含 `../` 等目录穿越字符
- 无路径规范化：没有使用 `normpath()` 或 `abspath()` 对路径进行规范化处理

3 漏洞 FL-001：动态页面路径穿越

3.1 漏洞位置

文件	app.py
函数	dynamic_page()
路由	GET /page
代码行	第 504-521 行（修复前）

3.2 漏洞原理

`dynamic_page()` 函数接受 URL 参数 `name` 后，直接将其与 "pages" 目录拼接：

```
page_path = os.path.join("pages", name)
```

`os.path.join()` 是 Python 的路径拼接函数。当第二个参数以 `../` 开头时，它会自动解析为上层目录。例如 `os.path.join("pages", "../app.py")` 的结果是 `pages/../app.py`，在文件系统中等价于 `app.py`。

攻击者可以通过精心构造 `name` 参数，使拼接后的路径指向项目目录之外的任意文件。由于代码没有对路径做任何检查就调用 `open()` 读取，攻击者可以获取服务器上的敏感文件内容。

3.3 漏洞利用过程

攻击步骤：

1. 攻击者访问 `/page?name=../app.py`
2. 服务器调用 `os.path.join("pages", "../app.py")` → 得到路径 `pages/../app.py`
3. `os.path.isfile("pages/../app.py")` 返回 `True`（因为 `app.py` 文件存在）
4. `open("pages/../app.py")` 读取了项目根目录下的 `app.py` 文件
5. 文件内容被返回到浏览器，攻击者获取到了项目源码

进一步利用：

- `/page?name=../../etc/passwd` → 读取 Linux 系统密码文件
- `/page?name=../../etc/shadow` → 读取用户密码哈希
- `/page?name=../../proc/self/environ` → 获取环境变量（可能含密钥）

3.4 文件访问流程

```
浏览器 (用户 : GET /page?name=../app.py)
↓ HTTP GET 请求
Flask 接收参数 (request.args.get("name", ""))
↓ name = "../app.py"
文件路径拼接 (os.path.join("pages", "../app.py"))
↓ page_path = "pages/../app.py" (等价于 "app.py")
读取文件 (open(page_path, "r"))
↓ 读取了项目根目录的 app.py
返回浏览器 (页面内容中显示了 app.py 源码)
↑
★ 漏洞产生位置 : 第 3 步和第 4 步之间没有任何路径校验
```

3.5 漏洞影响

- 源码泄露 : 攻击者可以读取项目中的所有 Python 文件 , 包括 app.py 和所有源代码
- 系统文件读取 : 通过 `../../etc/passwd` 可以读取 Linux 系统敏感文件
- 密钥泄露 : 通过 `../../proc/self/environ` 可能获取环境变量中的密钥
- 数据库文件 : 可以读取 data/users.db 获取所有用户数据

4 漏洞复现

4.1 实验环境

系统	Kali Linux
Web 服务	Flask 开发服务器 (127.0.0.1:5000)
测试工具	curl、Burp Suite

4.2 复现路径穿越

请求方式	GET
请求 URL	http://127.0.0.1:5000/page?name=../app.py
Payload	name=../app.py

```
# curl 测试命令
curl "http://127.0.0.1:5000/page?name=../app.py"

# 修复前结果：页面显示了 app.py 的源代码
# 响应中包含：Flask 配置、密钥、USERS 字典等敏感信息
```

[Burp Suite 请求/响应截图 — 路径穿越成功，读取 app.py 源码]

[浏览器页面截图 — 页面显示 app.py 文件内容]

4.3 复现读取系统文件

```
curl "http://127.0.0.1:5000/page?name=../../etc/passwd"

# 修复前结果：页面显示了 /etc/passwd 文件内容
# 包含：root、kali 等系统用户信息
```

[Burp Suite 请求/响应截图 — 读取 /etc/passwd]

[浏览器页面截图 — 页面显示 passwd 文件内容]

5 修复方案

5.1 修复 FL-001

修复前

```
@app.route("/page")
def dynamic_page():
    name = request.args.get("name", "")
    if not name:
        return redirect("/")

    page_path = os.path.join("pages", name)
    page_content = None

    if os.path.isfile(page_path):
        with open(page_path, "r", encoding="utf-8") as f:
            page_content = f.read()
    else:
        page_path_html = page_path + ".html"
        if os.path.isfile(page_path_html):
            with open(page_path_html, "r", encoding="utf-8") as f:
                page_content = f.read()

    if page_content is None:
        page_content = "页面不存在"
    ...
    return render_template("index.html", user=user, page_content=page_content)
```

修复后

```
ALLOWED_PAGES = {"help", "help.html", "about", "about.html", "index.html"}

@app.route("/page")
def dynamic_page():
    name = request.args.get("name", "")
    if not name:
        return redirect("/")

    if name not in ALLOWED_PAGES:
        page_content = "页面不存在"
    else:
        pages_dir = os.path.join(app.root_path, "pages")
        raw_path = os.path.join(pages_dir, name)
        real_path = os.path.normpath(raw_path)

        if not real_path.startswith(os.path.normpath(pages_dir) + os.sep) \
```



```

        and real_path != os.path.normpath(pages_dir):
            page_content = "页面不存在"
        elif os.path.isfile(real_path):
            with open(real_path, "r", encoding="utf-8") as f:
                page_content = f.read()
        else:
            page_content = "页面不存在"
    ...
    return render_template("index.html", user=user, page_content=page_content)

```

5.2 修复措施说明

措施一：白名单校验

定义 `ALLOWED_PAGES = {"help", "help.html", "about", "about.html", "index.html"}`。只有白名单中的页面名称允许访问，其他所有请求直接返回"页面不存在"。白名单是最有效的防御手段——任何不在预设列表中的路径都被拒绝。

措施二：路径规范化

使用 `os.path.normpath()` 对拼接后的路径进行规范化处理。例如 `os.path.normpath("pages/../app.py")` 会返回 `app.py`，暴露出路径穿越后的真实路径。然后与 `pages/` 目录的规范路径进行对比，确保最终路径仍在允许范围内。

措施三：固定目录前缀校验

使用 `real_path.startswith(pages_dir + os.sep)` 确保最终路径以 `pages` 目录的绝对路径开头。这样即使路径穿越成功，规范后的目录仍然不在 `pages/` 内，会被拒绝访问。

措施四：统一错误信息

所有拒绝访问的情况统一返回"页面不存在"，不区分"文件不存在"和"路径被拒绝"，攻击者无法通过错误信息推断服务器文件结构。

5.3 为什么能够防止文件包含漏洞

攻击者使用 `name=../app.py` 时：

1. `"../app.py"` 不在白名单 `ALLOWED_PAGES` 中 → 直接返回"页面不存在"，根本不会执行路径拼接和文件读取

即使白名单被绕过（例如后续维护中向白名单添加了 `*` 通配符），第二道防线也会生效：

1. `normpath("pages/../app.py")` → `"app.py"`
2. `normpath("pages") + "/"` → `"/root/flask_user_platform/pages/"`
3. `"app.py".startswith("../pages/")` → `False`，拒绝访问

6 修复前后对比

对比项	修复前	修复后
文件访问方式	用户输入直接拼接到路径	白名单限定 + 路径规范化
用户输入校验	无校验，任何值均可传入	必须在白名单 ALLOWED_PAGES 中
路径规范化	未使用 normpath()	使用 normpath() 规范化路径
目录穿越防护	无防护，../ 可直接穿越	startswith(pages_dir) 双重校验
错误提示	内容为空或直接报错	统一返回"页面不存在"
正常访问 help 页面	正常显示	正常显示 
源代码泄露风险	可读取 app.py	已防御 

7 修复验证

7.1 验证方法

使用与漏洞复现阶段相同的 Payload 对修复后的系统进行逐一测试。

7.2 验证结果

测试项	Payload	修复前结果	修复后结果	是否符合预期
正常页面	?name=help	显示帮助中心	显示帮助中心	✓
路径穿越	?name=../app.py	读取 app.py 源码	页面不存在	✓
深层穿越	?name=../../etc/passwd	读取系统文件	页面不存在	✓
反斜杠	?name=..\\app.py	路径穿越成功	页面不存在	✓
绝对路径	?name=/etc/passwd	可能读取文件	页面不存在	✓
不存在文件	?name=nonexist	无内容或报错	页面不存在	✓
空参数	?name=	无处理	跳转首页	✓

7.3 验证结论

- 所有路径穿越尝试（../、../../、..\、绝对路径）全部被拦截 ✓
- 正常页面访问（help）功能正常 ✓
- 不存在的页面返回统一错误信息 ✓
- 空参数处理正确 ✓

8 实验总结

8.1 文件包含漏洞产生原因

- 用户输入的 name 参数直接拼接到文件路径中，未做任何校验
- `os.path.join()` 自动解析 `../`，导致了目录穿越的可能性
- 没有限制文件访问的范围，任何路径下的文件都可以被读取

8.2 修复方法

防护措施	实现方式
白名单	ALLOWED_PAGES 集合限定可访问的页面名称
路径规范化	<code>os.path.normpath()</code> 规范化路径，暴露穿越行为
目录校验	<code>startswith(pages_dir)</code> 确保最终路径在允许范围内
统一错误	拒绝访问时返回"页面不存在"，不泄露路径信息

8.3 修复效果

- 路径穿越攻击 (`../`、`../../etc/passwd`) → 全部拦截 
- 绝对路径攻击 (`/etc/passwd`) → 拦截 
- 正常页面访问 (`help`) → 正常显示 
- 项目源码、系统敏感文件 → 无法读取 

8.4 实验收获

1. 理解了文件包含漏洞的产生原理：用户输入拼接路径后未校验
2. 掌握了白名单 + 路径规范化 + 目录校验的多层防御方法
3. 理解了 `os.path.join()` 和 `os.path.normpath()` 的行为特性
4. 通过修复前后对比验证，确认防御措施有效